JAVA COLLECTIONS FULL HANDBOOK

1. COLLECTIONS OVERVIEW

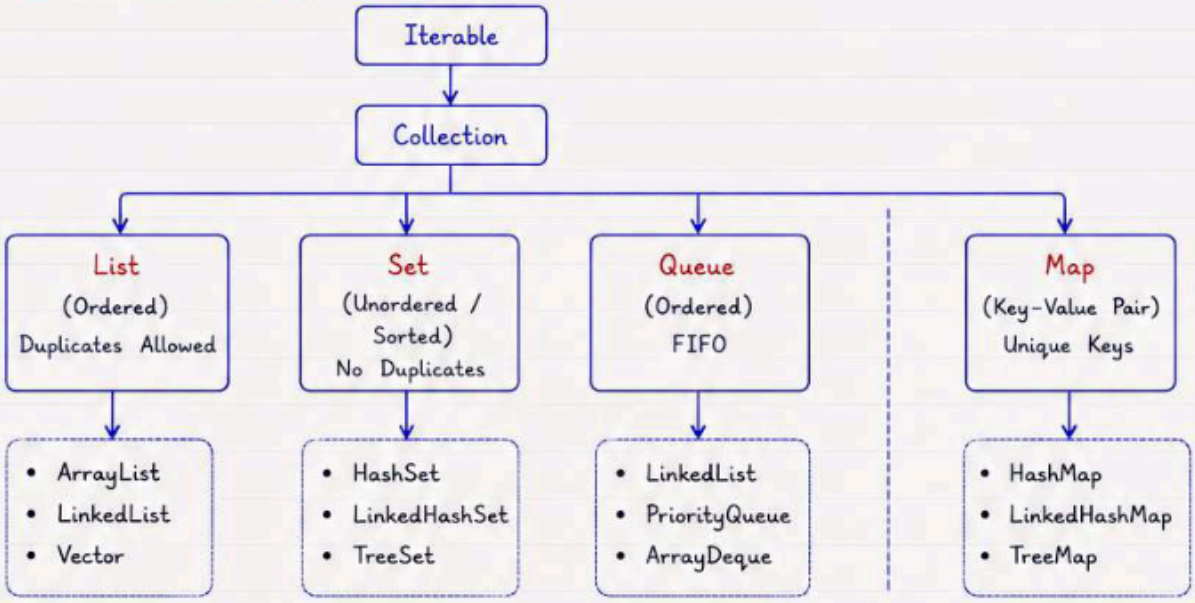
★ What is Collections Framework ?

Java Collections Framework is a architecture that provides ready-made classes and interfaces to store and manipulate groups of objects.

Why Collections ?

- To store multiple objects.
- To manipulate and process data.
- To reduce manual resizing.
- To provide better performance and easy to maintain code.

★ Collections Hierarchy



★ Quick Comparison

Feature	List	Set	Queue	Map
Order	Maintains Order	No Order / Sorted (TreeSet)	Maintains Order (FIFO)	No Order (Key-Value)
Duplicates	Allowed	Not Allowed	Allowed	Keys - Not Allowed Values - Allowed
Null Elements	Multiple Nulls Allowed	1 Null Allowed (HashSet)	1 Null Allowed (LinkedList)	1 Null Key (HashMap) Multiple Null Values
Access	Index Based	No Index	Sequential Access	Key Based
Performance	Fast Access	Fast Search (HashSet)	Depends on Implementation	Fast Lookup (HashMap)
Use Case	Store Ordered Collection	Store Unique Elements	Process in Sequence	Store Key-Value Data

★ Key Takeaway

- Use List when you need ordered collection with duplicates.
- Use Set when you need only unique elements.
- Use Queue when you need to process elements in sequence.
- Use Map when you need to store data in key-value pair.

Collections Framework



Powerful, Flexible & Easy to Use

Write Less Code ♥ Do More Work ♥

JAVA COLLECTIONS FULL HANDBOOK

2. LIST INTERFACE

What is List ?

List is an ordered collection that allows duplicate elements. Elements are stored in a specific order and can be accessed by their index.

List Example

0	1	2	3	4
10	20	20	30	40

Duplicates Allowed, Order Maintained, Index Based

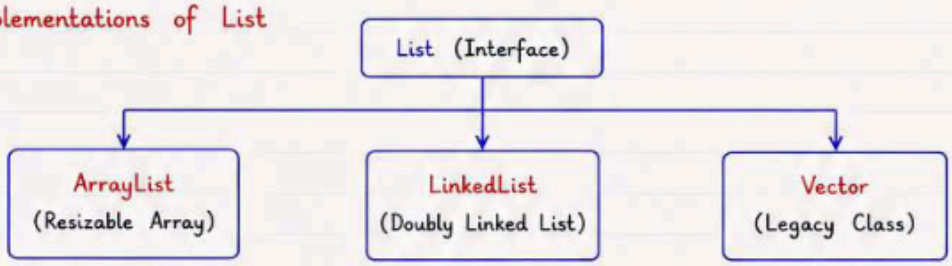
Features of List

- Maintains insertion order.
- Allows duplicate elements.
- Allows null values.
- Elements can be accessed by index.
- Useful when order of elements is important.

Common Methods

- add(E e) → Adds element
- add(int index, E e) → Adds at specific index
- get(int index) → Returns element
- set(int index, E e) → Updates element
- remove(int index) → Removes element
- contains(Object o) → Checks presence
- size() → Returns size

Implementations of List



List Implementations Comparison

Feature	ArrayList	LinkedList	Vector
Underlying Data Structure	Resizable Array	Doubly Linked List	Resizable Array
Order	Maintains Insertion Order	Maintains Insertion Order	Maintains Insertion Order
Duplicates	Allowed	Allowed	Allowed
Null Elements	Allowed	Allowed	Allowed
Random Access	Fast (O(1))	Slow (O(n))	Fast (O(1))
Insertion/Deletion	Slow in middle (O(n))	Fast (O(1))	Slow in middle (O(n))
Thread Safe	No	No	Yes
Best Use	More Read Operations	More Insert/Delete Operations	Legacy (Thread Safe) Requirements

Example (ArrayList)

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(20);  // duplicates allowed
list.add(30);
System.out.println(list.get(2));  // 20
System.out.println(list);         // [10, 20, 20, 30]
```

Use List when :

- You need ordered collection.
- You need to allow duplicates.
- You need index based access.

JAVA COLLECTIONS FULL HANDBOOK

3. ARRAYLIST

★ What is ArrayList ?

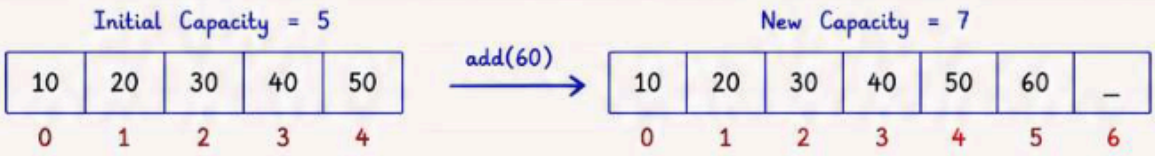
ArrayList is a resizable-array implementation of the List interface. It uses a dynamic array to store elements and grows automatically when the capacity is full.

Key Features

- Implements List interface.
- Ordered collection (maintains insertion order).
- Allows duplicate elements.
- Allows null elements.
- Not synchronized (not thread-safe).
- Fast random access (index based).

★ Internal Working

ArrayList internally uses a dynamic array. When the array becomes full, a new bigger array is created (usually 1.5 times) and all elements are copied to the new array.



★ Time Complexity

Operation	Complexity
get(index)	O(1)
add(E e) (at end)	O(1) amortized
add(index, E e)	O(n)
remove(index)	O(n)
contains(Object o)	O(n)
size()	O(1)

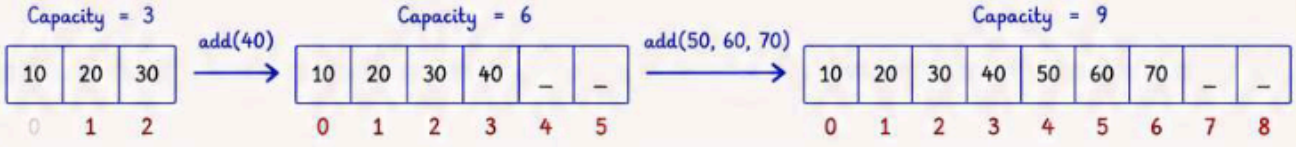
★ When to Use ArrayList ?

- When you need fast access to elements by index.
- When insertions and deletions are few compared to reads.
- When you need an ordered collection that allows duplicates.

★ Example

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);
list.add(1, 15);    // insert 15 at index 1
System.out.println(list);  // [10, 15, 20, 30]
System.out.println(list.get(2)); // 20
```

★ Resizing in ArrayList



Advantages

- ✓ Fast random access.
- ✓ Automatically resizable.
- ✓ Works well for read-intensive operations.
- ✓ Allows duplicate and null elements.

Disadvantages

- ✗ Slow insertions/deletions in middle (shifting required).
- ✗ Extra memory used during resizing.
- ✗ Not thread-safe.

JAVA COLLECTIONS FULL HANDBOOK

4. LINKEDLIST

What is LinkedList ?

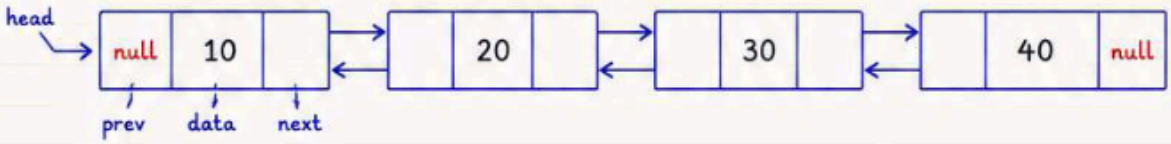
LinkedList is a part of the List interface.
It is implemented using a doubly linked list.
It allows insertion and deletion of elements at both ends and in between.

Key Features

- Maintains insertion order.
- Allows duplicate and null elements.
- Good performance for frequent insertions/deletions.
- Slower random access (not index based).
- Can be used as List, Queue and Deque.

Internal Working

LinkedList is a doubly linked list where each element (node) contains reference to both previous and next node.



Common Methods

add(E e)	Adds element at the end.
addFirst(E e)	Adds element at the beginning.
addLast(E e)	Adds element at the end.
removeFirst()	Removes and returns first element.
removeLast()	Removes and returns last element.
get(int index)	Returns element at index (slow).
set(int index, E e)	Updates element at index.
contains(Object o)	Checks presence of element.
size()	Returns number of elements.

LinkedList vs ArrayList

Feature	ArrayList	LinkedList
Data Structure	Resizable Array	Doubly Linked List
Access	Fast (index based)	Slow (index access)
Insert/Delete	Slow in middle O(n)	Fast O(1)
Memory Usage	Less	More (extra pointers)
Cache Performance	Better	Poorer
Use Case	More read operations	More insert/delete operations

Time Complexity

Operation	Complexity
addFirst(), addLast()	O(1)
add(index, E e)	O(n)
removeFirst(), removeLast()	O(1)
remove(index)	O(n)
get(index)	O(n)
set(index, E e)	O(n)
contains(Object o)	O(n)
size()	O(1)

Example

```
LinkedList<String> list = new LinkedList<>();
list.add("Aman");
list.add("Rahul");
list.add("Neha");
list.addFirst("Start");      // [Start, Aman, Rahul, Neha]
list.addLast("End");         // [Start, Aman, Rahul, Neha, End]
list.removeFirst();          // removes Start
list.removeLast();           // removes End
System.out.println(list);    // [Aman, Rahul, Neha]
System.out.println(list.get(1));   // Rahul
```

When to Use LinkedList ?

- When you need frequent insertions and deletions at start, end or middle.
- When you need to implement stack, queue or deque.
- When memory is not a major constraint.

Advantages

- ✓ Fast insertions and deletions.
- ✓ Can be used as List, Queue and Deque.
- ✓ Maintains insertion order.

Disadvantages

- ✗ Slow random access.
- ✗ More memory usage.
- ✗ Poor cache performance.

JAVA COLLECTIONS FULL HANDBOOK

5. SET INTERFACE

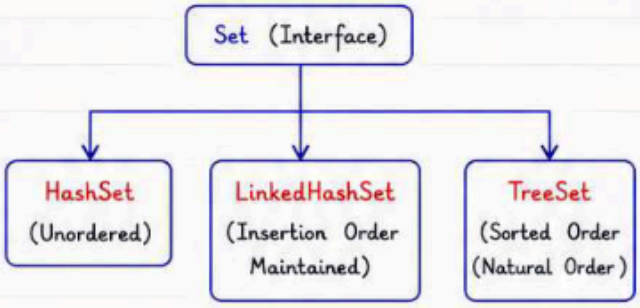
What is Set ?

Set is a collection that does not allow duplicate elements. It is used to store unique elements only.

Key Features

- Does not allow duplicate elements.
- At most one null element allowed (depends on implementation).
- Not index based.
- Used to store unique elements.
- Performance depends on the implementation.

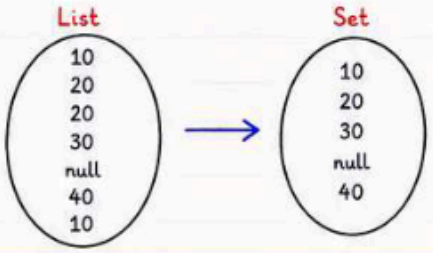
Set Implementations



Example

Input List : [10, 20, 20, 30, null, 40, 10]

Set (HashSet) : {10, 20, 30, null, 40}



Comparison of Set Implementations

Feature	HashSet	LinkedHashSet	TreeSet
Order	No Order	Insertion Order Maintained	Sorted Order (Natural Order)
Duplicates	Not Allowed	Not Allowed	Not Allowed
Null Element	1 Null Allowed	1 Null Allowed	Not Allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance	Very Fast	Fast	Slower (O(log n))
Use Case	When order is not important	When insertion order is required	When sorted order is required

Common Methods

- add(E e) → Adds element
- remove(Object o) → Removes element
- contains(Object o) → Checks if element exists
- size() → Returns size
- isEmpty() → Checks if empty
- clear() → Removes all elements
- iterator() → Returns an iterator

Example (HashSet)

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(20);    // duplicate, will not be added
set.add(null);  // one null allowed
set.add(30);
set.add(10);    // duplicate, will not be added
System.out.println(set);  // Output: [null, 20, 10, 30]
```



Key Takeaway

- Use HashSet when you need fast performance and order is not important.
- Use LinkedHashSet when you need unique elements and insertion order.
- Use TreeSet when you need unique elements in sorted order.

JAVA COLLECTIONS FULL HANDBOOK

6. MAP INTERFACE

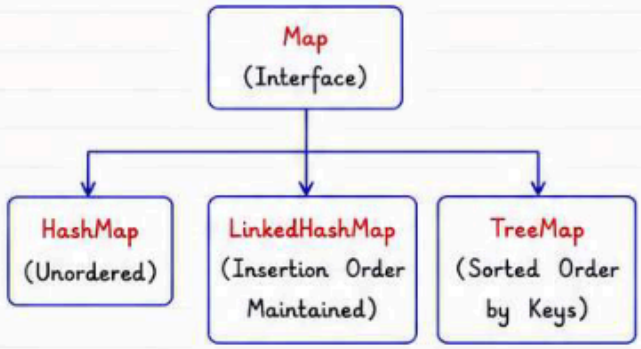
What is Map ?

Map is an interface in Java Collections that stores data in key-value pairs. Keys are unique, but values can be duplicate. Each key maps to exactly one value.

Key Features

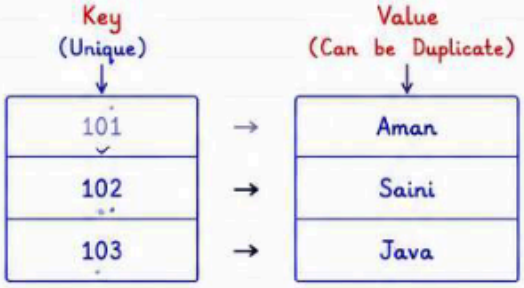
- Stores data in key-value pairs.
- Keys are unique, values can be duplicate.
- Each key maps to one value.
- At most one null key allowed (in some implementations).
- Multiple null values allowed.
- Not thread-safe (depends on implementation).

Map Implementations



Example

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Aman");
map.put(102, "Saini");
map.put(103, "Java");
System.out.println(map);
// Output: {101=Aman, 102=Saini, 103=Java}
```



Common Methods

- `put(K key, V value)` → Inserts or updates entry
- `get(Object key)` → Returns value for the key
- `remove(Object key)` → Removes entry for the key
- `containsKey(Object key)` → Checks if key exists
- `containsValue(Object value)` → Checks if value exists
- `size()` → Returns number of key-value pairs
- `isEmpty()` → Checks if map is empty
- `clear()` → Removes all entries
- `keySet()` → Returns set of keys
- `values()` → Returns collection of values
- `entrySet()` → Returns set of key-value pairs

Comparison of Map Implementations

Feature	HashMap	LinkedHashMap	TreeMap
Order	No Order	Insertion Order Maintained	Sorted Order (by Keys)
Null Key	1 Null Key Allowed	1 Null Key Allowed	Not Allowed
Null Values	Multiple Nulls Allowed	Multiple Nulls Allowed	Multiple Nulls Allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance	Very Fast	Fast	Slower (O(log n))
Use Case	When order is not required	When insertion order is required	When sorted order of keys is required

When to Use Map ?

- ✓ Use Map when you need to store and access elements using a unique key.
- ✓ Use HashMap for best performance when order is not important.
- ✓ Use LinkedHashMap when you need insertion order.
- ✓ Use TreeMap when you need sorted keys.

Think of Map as a Dictionary where each word (key) has a meaning (value).

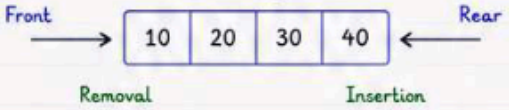
JAVA COLLECTIONS FULL HANDBOOK

7. QUEUE, PRIORITYQUEUE & STACK

1. QUEUE (Interface)

Queue follows FIFO (First In First Out) order.
Elements are added at the rear and removed from the front.

Queue Diagram



Implementations

- LinkedList → Most common Queue implementation
- ArrayDeque → Better performance, resizable array (recommended)
- PriorityQueue → Elements ordered by priority

Common Methods

- offer(E e) → Inserts element
- poll() → Retrieves and removes head
- peek() → Retrieves head without removing
- isEmpty() → Checks if queue is empty
- size() → Returns number of elements

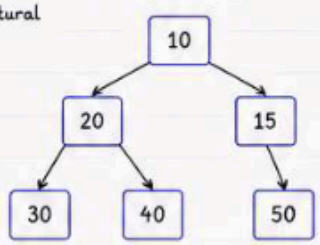
Example (Using LinkedList)

```
Queue<Integer> queue = new LinkedList<>();
queue.offer(10);     // add
queue.offer(20);
queue.offer(30);
System.out.println(queue.peek());   // 10
System.out.println(queue.poll());   // 10 (removed)
System.out.println(queue);          // [20, 30]
```

2. PRIORITYQUEUE

PriorityQueue orders elements based on their natural ordering or a custom Comparator.
The head is the smallest element by default (Min-Heap).

Min-Heap Diagram



Example

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10); pq.offer(20); pq.offer(40);
System.out.println(pq);          // [10, 30, 20, 40] (order may vary)
System.out.println(pq.poll()); // 10 (smallest element)
```

Common Methods

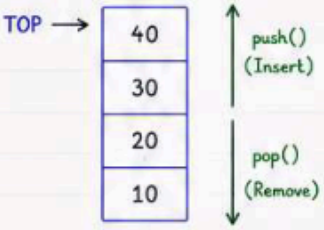
- offer(E e) → Inserts element
- poll() → Retrieves & removes head (highest priority)
- peek() → Retrieves head without removing
- size() → Returns size
- isEmpty() → Checks if empty

Note : PriorityQueue does NOT maintain insertion order.
Elements are stored based on priority (natural order or comparator).

3. STACK

Stack follows LIFO (Last In First Out) order.
The last element inserted is the first one removed.

Stack Diagram



Common Methods

- push(E e) → Inserts element
- pop() → Removes top element
- peek() → Retrieves top without removing
- isEmpty() → Checks if empty
- search(E e) → Returns position of element from top

Example (Using Deque / ArrayDeque)

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.peek());   // 30
System.out.println(stack.pop());    // 30
System.out.println(stack);          // [20, 10]
```

Note :
- Stack class is legacy. Prefer Deque (ArrayDeque).
- Deque can be used as Stack or Queue.

Quick Comparison

Feature	Queue	PriorityQueue	Stack
Order	FIFO (First In First Out)	Based on Priority (Min-Heap by default)	LIFO (Last In First Out)
Insertion	At Rear	Anywhere	At Top
Deletion	From Front	From Head (Highest Priority)	From Top
Implementation	LinkedList / ArrayDeque	PriorityQueue	Deque (ArrayDeque)
Use Case	Task scheduling, BFS	Task scheduling by priority, Dijkstra's Algorithm	Backtracking, Undo operation, Expression evaluation

JAVA COLLECTIONS FULL HANDBOOK

8. HASHMAP vs LINKEDHASHMAP vs TREEMAP

★ Overview

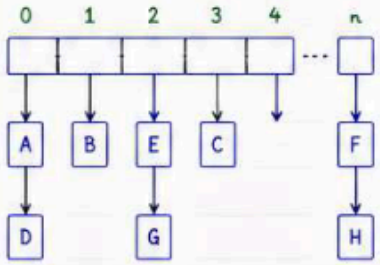
All three classes implement **Map** interface and store data in **key-value pairs**, but they differ in ordering, null handling and internal structure.

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order (Random)	Insertion order maintained	Sorted order by keys (Natural order or Comparator)
Null Key	1 null key allowed	1 null key allowed	No null key
Null Values	Multiple null values allowed	Multiple null values allowed	Multiple null values allowed
Underlying Data Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Performance (get/put)	Very fast (average O(1))	Fast (average O(1))	Slower (O(log n))
Use Case	When order is not important and we need best performance	When insertion order needs to be maintained (e.g. caching, LRU)	When keys need to be sorted (range operations, navigable map)

★ Internal Structure (Visual)

HashMap (No Order)

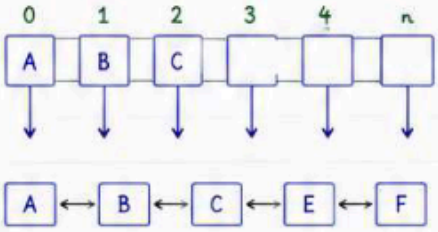
Bucket Array



Elements stored in buckets (No order guarantee)

LinkedHashMap (Insertion Order)

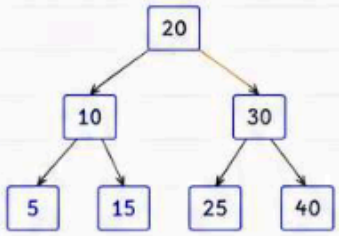
Bucket Array



Hash Table + Doubly Linked List (maintains insertion order)

TreeMap (Sorted Order)

Red-Black Tree



Keys are always sorted (Natural Order / Comparator)

★ Example (Same Input)

Input Insert Order : 30, 10, 20, 40, 5

HashMap (No Order)	LinkedHashMap (Insertion Order)	TreeMap (Sorted Order by Keys)
{30=Thirty, 10=Ten, 20=Twenty, 40=Forty, 5=Five}	{30=Thirty, 10=Ten, 20=Twenty, 40=Forty, 5=Five}	{5=Five, 10=Ten, 20=Twenty, 30=Thirty, 40=Forty}
(Order not guaranteed)	(Insertion order maintained)	(Sorted by keys)

★ Key Points

- **HashMap** is fastest.
- **LinkedHashMap** maintains insertion order with slight extra memory.
- **TreeMap** keeps keys sorted and is useful for range queries (firstKey(), lastKey(), subMap()).
- Choose collection based on order requirement & performance.

★ Quick Summary

Use HashMap when :	Use LinkedHashMap when :	Use TreeMap when :
✓ Order is not important	✓ You need insertion order	✓ You need sorted keys
✓ Fastest performance needed	✓ Useful in caching (LRU Cache)	✓ Range / navigation operations
✓ Default choice for most cases	✓ Slightly more memory than HashMap	✓ Slower but sorted data